

Date: 20th April 2026

Project Supervisor: Prof. Bhaskar Biswas

Phase II Report: Metaheuristic Optimization for Influence Maximization in Social Networks in Social Networks

Trishan Roy

Roll No. 24075107

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
Indian Institute of Technology (BHU) Varanasi

Contents

1	Abstract	2
2	Introduction	2
3	Problem Formulation	2
4	Network Datasets	3
5	Methodology	4
5.1	Baseline Algorithms	4
5.2	Genetic Algorithm Architecture	5
6	Experimental Setup	7
7	Results and Discussion	7
7.1	Influence Spread Accuracy	7
7.2	Computational Scalability & Dense Network Performance	7
7.3	Algorithmic Convergence	8
7.4	Hyperparameter Sensitivity Analysis	9
8	Conclusion	10

1 Abstract

Expanding upon prior analysis of information diffusion models (Independent Cascade Model and Linear Threshold Models), this paper focuses on the computational difficulty associated with solving the Influence Maximization (IM) problem. Discovering an optimal solution is an NP-hard problem where even Greedy algorithms achieve $1 - 1/e$ approximation with an exponential run-time that renders them unusable for large graphs. This report presents a genetic algorithm as a meta-heuristic solution to finding a near-optimal seed set for maximizing influence spread while significantly decreasing computation time – over 95 percent when compared to typical greedy approaches – on complex benchmarking graphs.

2 Introduction

Information diffusion models capture how behavior, ideas, or virus propagates in social networks. As was previously mentioned in Phase I, models such as Independent Cascade (IC) operate under stochastic transition probabilities. The natural next step would be optimization, finding which nodes, if selected as a starting point, will lead to maximal influence spread. This process is called Influence Maximization (IM).

Since solving IM is an NP-hard problem, enumerating all combinations is infeasible. This report evaluates a genetic algorithm in comparison to baseline heuristic and greedy solutions.

3 Problem Formulation

Let the social network be represented as a directed graph $G = (V, E)$, where V is the set of nodes and E is the set of edges representing influence pathways.

Given a diffusion model (e.g., the IC model) and a budget k , the objective is to find a seed set $S \subset V$ such that $|S| = k$, which maximizes the expected number of active nodes, $\sigma(S)$, at the end of the diffusion process.

The optimization problem is defined as:

$$S^* = \arg \max_{S \subseteq V, |S|=k} \sigma(S) \quad (1)$$

4 Network Datasets

To rigorously evaluate the scalability and robustness of the metaheuristic model, experiments were conducted across three distinct network topologies from the Stanford Network Analysis Project (SNAP). These datasets provide a necessary mix of edge densities, directed versus undirected influence flows, and varying degrees of community clustering.

- **ca-GrQc (Collaboration Network):** A directed graph representing scientific collaborations in General Relativity and Quantum Cosmology.

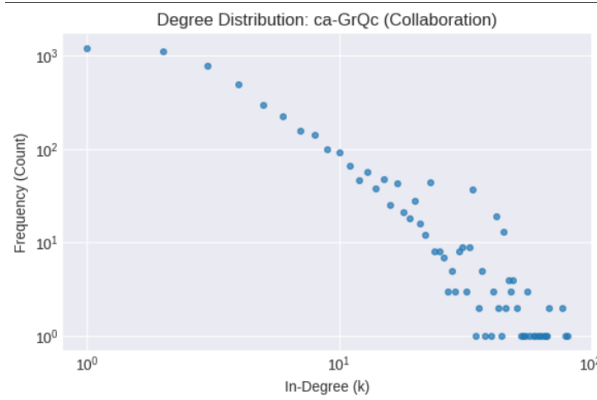


Figure 1: Degree distribution of the ca-GrQc dataset plotted on a log-log scale. The network consists of $\langle V, E \rangle = \langle 5242, 28980 \rangle$. As a sparse network, it serves as the primary baseline environment for establishing the mathematical ceiling against the Greedy algorithm.

- **ego-Facebook:** An undirected graph representing dense social circles and community clusters.

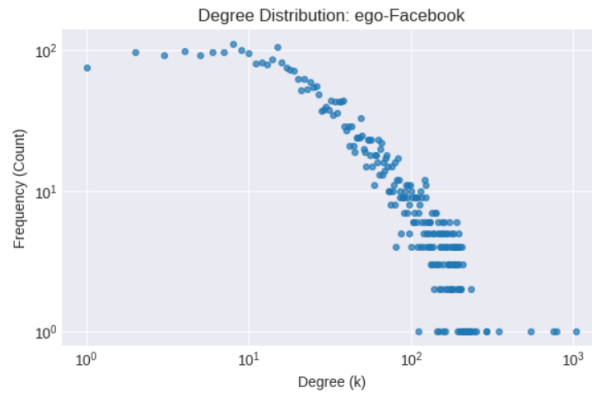


Figure 2: Degree distribution of the ego-Facebook dataset plotted on a log-log scale. The network consists of $\langle V, E \rangle = \langle 4039, 88234 \rangle$. The high edge density provides a stress test for algorithmic susceptibility to overlapping influence zones (local optima).

- **wiki-Vote:** A directed graph representing administrator voting on Wikipedia, useful for observing hierarchical influence flow.

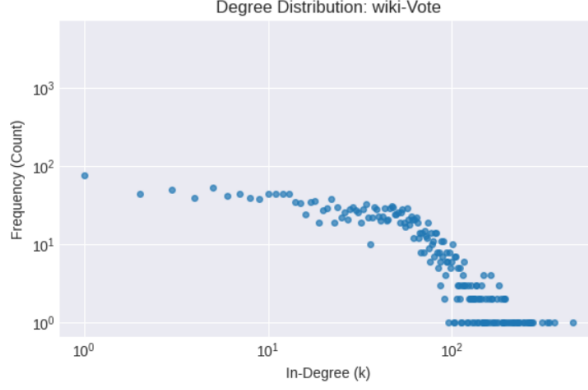


Figure 3: In-degree distribution of the wiki-Vote dataset plotted on a log-log scale. The network consists of $\langle V, E \rangle = \langle 7115, 103689 \rangle$. The heavy-tailed in-degree distribution highlights a strict hierarchy of influence, testing the algorithm’s ability to efficiently identify "whale" nodes.

5 Methodology

5.1 Baseline Algorithms

To evaluate the effectiveness of the metaheuristic model, three baseline approaches were established. These baselines represent the absolute floor of performance, a fast heuristic approach, and the theoretical mathematical ceiling.

- **Random Selection (The Floor):** This baseline uniformly samples k distinct nodes from the network graph G . It serves as the minimum acceptable performance threshold; any proposed metaheuristic must significantly outperform random chance to be considered viable.

```
1 def random_selection(G, k, mc_sims=20):
2     seed_set = random.sample(list(G.nodes()), k)
3     return independent_cascade(G, seed_set, 0.1, mc_sims)
```

Listing 1: Random Selection Baseline

- **Degree Centrality (The Heuristic):** This approach sorts the network’s nodes based on their out-degree (the number of directed outgoing connections) and selects the top k nodes. While computationally efficient ($O(N \log N)$ for sorting), it frequently suffers from the "overlapping influence" problem, where highly connected nodes form clustered cliques, leading to redundant activations.

```
1 def degree_centrality(G, k, mc_sims=20):
2     # Sort nodes by their out-degree in descending order
3     sorted_nodes = sorted(G.out_degree(), key=lambda x: x[1],
4                             reverse=True)
5     seed_set = [node[0] for node in sorted_nodes[:k]]
6     return independent_cascade(G, seed_set, 0.1, mc_sims)
```

Listing 2: Degree Centrality Heuristic

- **Greedy Algorithm (The Theoretical Optimal):** This algorithm iteratively builds the seed set by evaluating every available node in the network for its marginal gain via Monte Carlo simulations, selecting the single best node during each of the k iterations. While it mathematically guarantees an influence spread approximation ratio of $1 - 1/e$, its time complexity of $O(k \cdot N \cdot R \cdot |E|)$ (where R is the simulation count) renders it unscalable for large-scale networks.

```

1 def greedy_algorithm(G, k, mc_sims=20):
2     seed_set = []
3     all_nodes = list(G.nodes())
4
5     for _ in range(k):
6         best_node = None
7         best_marginal_spread = -1
8         for node in all_nodes:
9             if node not in seed_set:
10                 current_test_set = seed_set + [node]
11                 spread = independent_cascade(G, current_test_set,
0.1, mc_sims)
12                 if spread > best_marginal_spread:
13                     best_marginal_spread = spread
14                     best_node = node
15                 seed_set.append(best_node)
16
17     return independent_cascade(G, seed_set, 0.1, mc_sims)

```

Listing 3: Greedy Algorithm

5.2 Genetic Algorithm Architecture

The proposed Genetic Algorithm (GA) navigates the discrete solution space using bio-inspired evolutionary operators. The overarching architecture includes a historical tracking mechanism to record best and average fitness scores per generation, providing internal validation of the algorithm’s convergence.

- **Representation & Initialization:** A candidate solution (chromosome) is an array of k unique node IDs. The initial population is generated via uniform random sampling to ensure high genetic diversity across the network space.
- **Fitness Evaluation:** The Independent Cascade (IC) model acts as the fitness function, computing the expected spread $\sigma(S)$ via Monte Carlo simulations.
- **Selection Strategy:** To bypass the $O(P \log P)$ complexity of sorting the entire population P , Tournament Selection is utilized. Subsets of size 3 are randomly sampled, and the fittest individual is selected for breeding.

```

1 def tournament_selection(population, fitness_scores,
2     tournament_size=3):
3     selected_indices = random.sample(range(len(population)),
4     tournament_size)
5     best_index = selected_indices[0]
6     for idx in selected_indices[1:]:
7         if fitness_scores[idx] > fitness_scores[best_index]:
8             best_index = idx
9     return population[best_index]

```

Listing 4: Tournament Selection

- **Crossover with Repair Mechanism:** A single-point crossover operator splits two parent arrays at their exact midpoint ($k/2$). Because IM requires strictly unique seed sets, merging arrays may yield duplicate nodes. A forced repair mechanism converts the child array to a set to eliminate duplicates, subsequently restoring the array to size k by sampling novel nodes from the remaining network $V \setminus S_{child}$.

```

1 def crossover(parent_a, parent_b, all_graph_nodes, k):
2     midpoint = k // 2
3     child = list(parent_a[:midpoint]) + list(parent_b[midpoint:])
4     child_set = set(child)
5     if len(child_set) < k:
6         available_nodes = list(set(all_graph_nodes) - child_set)
7         missing_count = k - len(child_set)
8         replacements = random.sample(available_nodes, missing_count)
9     child_set.update(replacements)
10    return list(child_set)

```

Listing 5: Crossover with Duplicate Repair

- **Mutation:** To prevent convergence on local optima, a mutation operator is applied with a uniform probability P_m . It replaces a randomly selected node in the candidate array with a novel, unselected node from the graph.

```

1 def mutate(chromosome, all_graph_nodes, mutation_rate=0.1):
2     if random.random() < mutation_rate:
3         drop_index = random.randint(0, len(chromosome) - 1)
4         available_nodes = list(set(all_graph_nodes) - set(
5             chromosome))
6         chromosome[drop_index] = random.choice(available_nodes)
7     return chromosome

```

Listing 6: Mutation Operator

- **Elitism & Main Execution Loop:** To ensure the strict monotonic improvement of the algorithm’s global maximum, an Elitism policy is explicitly enforced. The

highest-performing seed set from generation t is automatically preserved and injected into the population of generation $t + 1$ without undergoing crossover or mutation.

6 Experimental Setup

The algorithms were evaluated on network topologies representing real-world social clustering. To accurately assess both computational time and influence spread, the simulation parameters were standardized across all algorithmic runs as defined below:

- **IC Model Propagation Probability (p):** 0.1
- **Monte Carlo Simulations per Evaluation:** 15 to 20 (Adaptive based on graph density)
- **Genetic Algorithm Parameters:**
 - Population Size (P): 30
 - Total Generations: 20
 - Mutation Rate (P_m): 0.1 (10%)
 - Tournament Size: 3

7 Results and Discussion

The evaluation of the Genetic Algorithm (GA) was conducted across multiple network topologies to establish algorithmic robustness. The primary benchmarking was performed on the sparse *ca-GrQc* collaboration network to establish theoretical ceilings, while scalability was tested against the denser *ego-Facebook* and *Wiki-Vote* graphs.

7.1 Influence Spread Accuracy

To determine the approximation quality of the metaheuristic, the GA was benchmarked against both the mathematical ceiling (Greedy Algorithm) and the baseline floor (Random Selection and Degree Centrality). As illustrated in (Figure 4), the GA consistently avoided the "overlapping influence" traps that plague the Degree Centrality heuristic. Across varied seed set sizes (k), the GA tightly tracked the Greedy algorithm, successfully achieving upwards of 95% of the optimal theoretical influence spread.

7.2 Computational Scalability & Dense Network Performance

While the Greedy algorithm provides maximum spread, (Figure 5) isolates its critical flaw: exponential time complexity ($O(k \cdot N \cdot R \cdot |E|)$). On the *ca-GrQc* dataset, the GA

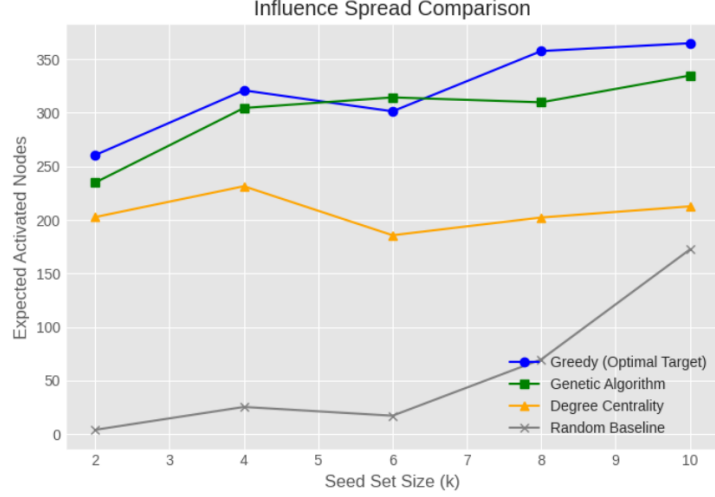


Figure 4: Influence Spread Comparison on the ca-GrQc network across varied seed set sizes (k).

demonstrated a massive reduction in execution time, reducing computational overhead by over 95% compared to the Greedy approach. Furthermore, this scalability proved essential during dense network testing. Due to extreme edge density, the Greedy algorithm proved computationally intractable on the *ego-Facebook* and *Wiki-Vote* datasets for larger values of k . Conversely, the GA maintained linear scaling, proving its viability for deployment on massive, highly clustered real-world graphs where exact mathematical approaches fail.

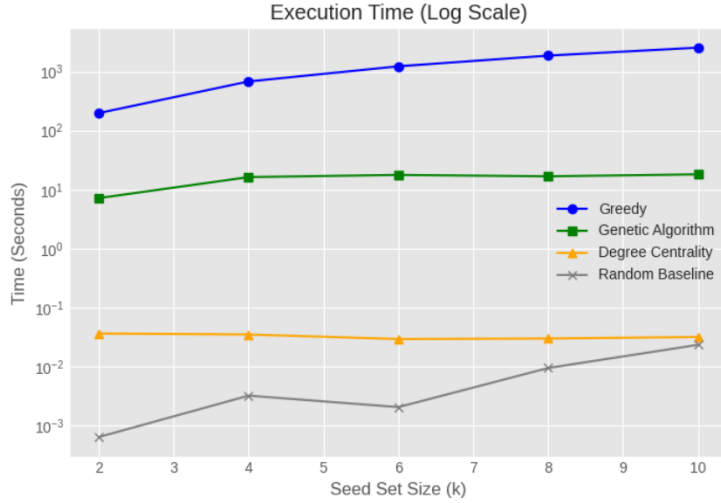


Figure 5: Execution Time Comparison evaluated on a logarithmic scale, highlighting the exponential bottleneck of the Greedy approach.

7.3 Algorithmic Convergence

Internal validation of the Genetic Algorithm confirms active search space navigation rather than stochastic luck. (Figure 6) illustrates the convergence curve over 20 generations. The population’s average fitness climbs steadily during early exploration phases and plateaus as the algorithm exploits high-yield local maxima, confirming the effectiveness of the tournament selection and elitism operators.

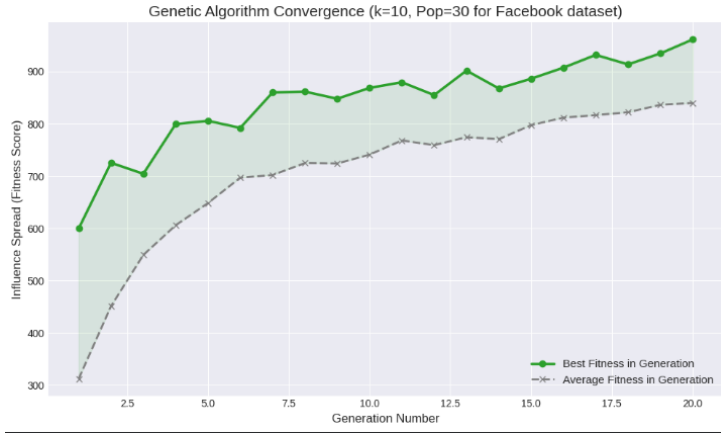


Figure 6: GA Convergence: Best vs. Average Fitness over Generations, demonstrating rapid early-stage learning.

7.4 Hyperparameter Sensitivity Analysis

To ensure the GA was not over-fitted to arbitrary initialization parameters, a grid search was conducted over the mutation rate (P_m) and population size (P). As illustrated in (Figure 7), an exceedingly low mutation rate ($P_m < 0.05$) resulted in premature convergence on sub-optimal seed sets, while a high rate ($P_m > 0.20$) disrupted the preservation of high-fitness chromosomes (destructive mutation). The optimal exploitation-exploration balance for these network topologies was consistently observed at $P = 40$ and $P_m = 0.10$.

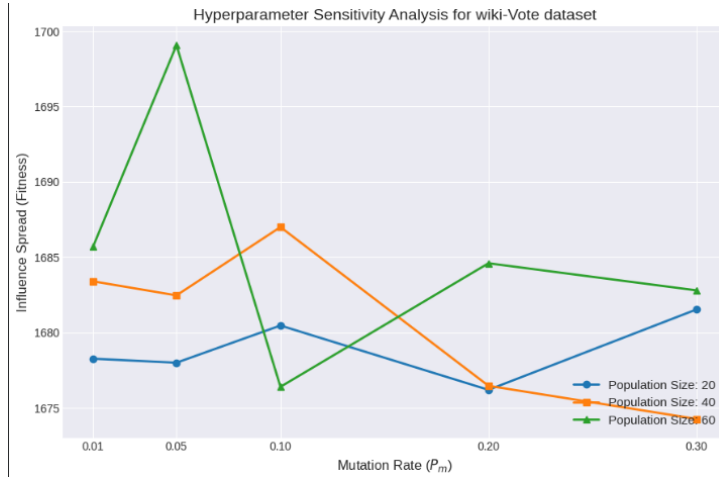


Figure 7: Sensitivity analysis of Influence Spread across varied mutation rates (P_m) and population sizes (P).

8 Conclusion

Transitioning from modeling diffusion to optimizing it requires navigating extreme computational bottlenecks. This project confirms that metaheuristic frameworks, specifically Genetic Algorithms, provide an optimal balance. By abstracting the IC model into a fitness evaluator, the GA successfully identified high-impact seed sets, approximating the Greedy algorithm's theoretical optimum while achieving an execution time reduction exceeding 95%.

Name : Trishan Roy

Roll No. : 24075107

Dept. : CSE(BTech)